



Guia de Formação Git/GitHub 25/26

FORMAÇÕES AEROTÉC

1.º Semestre 2025-2026

Contents

1	Introdução	3
1.1	O que é o <i>Git</i> ?	3
1.2	O que é o <i>GitHub</i> ?	3
2	Instalação de <i>Software</i>	4
3	Nomenclatura, Comandos de <i>Git</i> e Convenções	5
3.1	Alguns comandos	5
3.2	Comandos para <i>Commits</i> e Convenções	6
3.2.1	<i>Commit</i> Ideal	7
4	Aplicação de Comandos a um Projeto	8

1 Introdução

O presente documento constitui uma abordagem inicial para *Git* e *GitHub* bem como uma *cheatsheet* para as formações de Git/GitHub inseridas no contexto das formações obrigatórias de recrutamento do *Aerotéc*.

Este ficheiro é atualizado e a versão mais recente pode sempre ser encontrada no seguinte link:

Link Guia Git/GitHub

Última atualização: 22 de outubro de 2025

Gonçalo Rodrigues - Coordenador Web.Dev

1.1 O que é o *Git*?

Git é um sistema de controlo de versões que atua como um histórico que pode ser usado simultaneamente por varios utilizadores. É mais comum ser usado no contexto de desenvolvimento de código e portanto os exemplos adiante usarão uma linguagem de programação para simular o *workflow* de um trabalho em equipa.

Pela simplicidade serão usados *scripts* em *Python*, simples o suficiente para demonstrar o efeito. Vai ser usada a extensão *Code Runner* do *VS Code* para evitar qualquer compilador ou interpretador e consequentemente qualquer *download* adicional.

O *Git* é um programa local, gratuito e *open source*, isto é, cujo código é público e acessível, de livre utilização e distribuição.

De uma forma geral, ele atua quase que tirando "fotografias" ao código existente quando executados os devidos comandos e mantendo registos de todas e quaisquer atualizações que advém destes processos.

Em qualquer projeto, muitas vezes, tentamos criar novas funcionalidades ou implementar novas ideias que mais tarde não resultam. É difícil voltar a uma versão "boa" anterior, principalmente quando muitas alterações foram feitas posteriormente. O *Git* permite exatamente trocar as "más" versões pelas "boas" versões com relativa facilidade.

Num projeto simples com um único desenvolvedor é fácil correr o código, detetar erros e alterar, mas em projetos multidisciplinares, com vários membros, simultaneidade de trabalho e maior complexidade, existe a necessidade de que as alterações que fazemos no nosso código sejam refletidas no código de todos os restantes membros do projeto.

1.2 O que é o *GitHub*?

GitHub é uma plataforma *online* que gere e guarda remotamente, em *cloud*, repositórios do tipo *Git*, isto é, que usam este sistema de controlo de versões. É esta plataforma que relamente permite a colaboração e o trabalho simultâneo em equipas já que o *Git* apenas atua localmente.

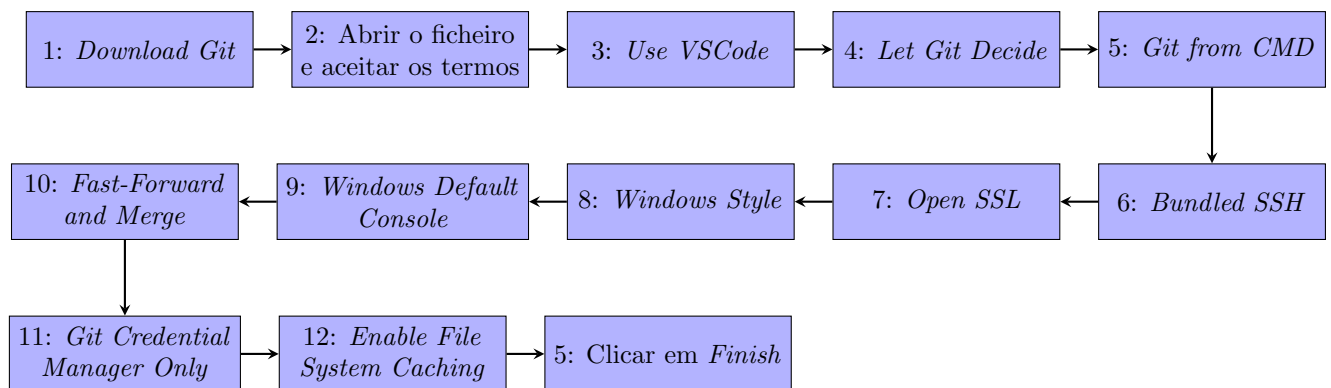
Permite também uma visualização daquilo que realmente está a ser feito, muitas das vezes "às cegas" na linha de comandos.

2 Instalação de *Software*

Para acompanhar melhor a formação ou mesmo para ajudar a explorar e perceber os comandos é recomendado o seguinte, por esta ordem:

- Criar conta no *GitHub*
- Transferir e instalar o *VSCode*
- Transferir e instalar o *Git* (ver detalhes abaixo)

De forma a uniformizar os comandos de *Git*, o modo geral como se apresentam e evitar conflitos entre inúmeras possíveis configurações segue o guia de instalação para *Windows 10/11*:



Depois da instalação, verifica-se se esta foi bem sucedida. Abrindo um novo terminal ou *Windows PowerShell*:

Git Comandos

```
git
```

```
# Deve gerar uma lista de comandos úteis.  
# Caso contrário a instalação não foi bem sucedida.
```

3 Nomenclatura, Comandos de *Git* e Convenções

Existem alguns termos que serão usados frequentemente em qualquer projeto em *Git* e que são enumerados abaixo. São apenas termos e não comandos.

- **Repositório (Repo):** pasta ou diretório do projeto que é seguido e controlado pelo *Git*. Existe distinção entre local (computador pessoal) e remoto (no GitHub, na *cloud*).
- **Commit:** Cria uma instância, uma versão ou uma "fotografia" de todo o projeto para determinado utilizador num momento específico.
- **Commit Hash:** Sequência longa de caracteres alfanuméricos que identificam o *commit*. Normalmente, dentro de um projeto, o *Git* usa apenas uma sequência mais curta de 7 caracteres, desde que o identificador seja único em todo o histórico de *commits*. Se houver muitos *commits* ou se a sequência curta for repetida (raro), o *Git* atualiza automaticamente para identificadores curtos de 8 caracteres.
- **Branch:** Linha de desenvolvimento. A estrutura de um projeto é, regra geral, ramificada e existe uma *branch* para cada *feature*, subprojeto ou utilizador.
- **Merge:** Combinação de duas *branches* numa única.
- **Pull Request:** Proposta de revisão para uma futura *merge*. Normalmente quando os códigos a ser intercalados foram desenvolvidos por pessoas diferentes, se está a publicar numa das *branches* principais ou quando a configuração da organização em que se está inserido assim o exige.

3.1 Alguns comandos

Abaixo segue uma lista não exaustiva dos comandos mais comuns usados no controlo de versões em *Git*.

git clone : Copia localmente um repositório remoto.

Transfere todos os ficheiros e histórico de atualizações do projeto.

Define o repositório remoto como *origin*.

Cria uma branch local que segue *origin/main* ou *origin/master*.

git fetch origin : Faz uma atualização dos ficheiros locais sem fazer qualquer *merge*.

Mantém os ficheiros locais atualizados sem alterar a branch atual.

git pull : Faz o mesmo que **fetch + merge**.

Realmente atualiza os ficheiros, intercalando ambas as versões. Se a *merge* for automática, isto é, não existirem conflitos, o código da *branch* de base tem prioridade.

git push : Envia *commits* locais para o repositório remoto.

Qualquer atualização ou comentário/descrição do *commit* está acessível no *GitHub* para todos os utilizadores com acesso ao repositório.

git revert <hash > : Faz um novo *commit* que desfaz um *commit* anterior.

Boa prática, preserva o histórico.

`git reset <hash >` : Move o ponteiro *HEAD* para um *commit* anterior identificado pela *hash*.
Usado em situações excepcionais, má pratica se usado regularmente, elimina o histórico.
Flags: `--soft` (*staged*) , `--mixed` (*unstaged*), `--hard` (*deletes*).

`git rebase <branch_name >` : Aplica os *commits* no *branch* atual no topo da *branch* especificada.
Não pode ser usada em *branches* comuns no desenvolvimento de vários utilizadores.
Apenas deve ser usado em *branches* pessoais.
Estrutura de *commits* linear em vez de ramificada. Melhora a legibilidade.

`git branch <branch_name >` : Cria uma nova *branch* com o nome especificado a partir da atual.

`git checkout <branch_name >` : Troca para a *branch* com o nome especificado.

`git checkout -b <branch_name >` : Cria uma *branch* com o nome especificado e troca da *branch* atual para esta.

`git checkout -b <new_branch > <base_branch >` : Cria uma *branch* com o nome especificado em primeiro lugar e que é uma cópia da *branch* especificada em segundo. Troca para esta nova *branch*.

`git checkout -b <new_branch > <base_branch >` : Cria uma *branch* com o nome especificado em primeiro lugar e que é uma cópia da *branch* especificada em segundo. Troca para esta nova *branch*.

`git branch -d <branch_name >` : Elimina uma *branch* localmente.

`git push origin --delete <branch_name >` : Elimina uma *branch* do repositório remoto.

`git merge <branch_name >` : Efetua uma *merge* da *branch* especificada para a *branch* atual.
Deve-se trocar par a *branch* de base previamente.

`git log --oneline` : Apresenta os *commits* efetuados com as *hashes* curtas.

3.2 Comandos para *Commits* e Convenções

Existem várias maneiras de criar *commits*. A mais fácil é, claramente, usar a *GUI* do *VSCode*, mas também é importante perceber como tal poderá ser feito inteiramente na linha de comandos. Abaixo estão alguns dos principais comandos usados neste sentido.

`git status` : Apresenta toda e qualquer alteração face ao *commit anterior*.

`git diff` : Apresenta uma lista de diferenças entre a versão atual e a do último *commit*.
Podem ser usadas duas *hashes* para especificar entre que *commits* se quer efetuar a comparação.

`git add` : *Stage* das alterações prontas a ser lançadas no *commit* seguinte.
No *VSCode* as alterações em *stage* abrem um *diff* quando clicadas.

`git restore --staged <relative_path >` : *Unstage* das alterações efetuadas.
As alterações não são perdidas, apenas organizadas de forma diferente indicando que não estão prontas para um *commit*.

`git commit -m "descrição curta"` : Cria um *commit* com a descrição indicada.
Existem outras maneiras que facilitam a introdução de uma descrição mais longas como por exemplo o `git commit`.

Embora não sejam estritamente obrigatórias, existem regras ou convenções a seguir na nomenclatura dos *commits* o que facilita o *debug* caso sejam encontrados erros. Alguns casos mais comuns aparecem na tabela a seguir.

Convenção	Descrição
<code>feat:</code>	Introduzir uma nova <i>feature</i>
<code>fix:</code>	Corrigir um <i>bug</i>
<code>docs:</code>	Adicionada ou alterada a documentação
<code>style:</code>	Formatação sem alteração do código em si. Mudança apenas visual para aumentar a legibilidade.
<code>refactor:</code>	Pequenas alterações sem novas <i>features</i> nem correção de <i>bugs</i>
<code>perf:</code>	Melhorias de <i>performance</i>
<code>test:</code>	Adicionar ou alterar testes do código
<code>chore:</code>	Manutenção e atualizações de dependências
<code>revert:</code>	Reverter um <i>commit</i> anterior

Table 1: Mensagens de *Commit* (Simplificadas)

3.2.1 *Commit* Ideal

Assim, o *commit* ideal seguirá o modelo abaixo. Entre parênteses estão apenas indicações do que implica cada linha.

Commit Ideal

<code>feat: Nova função de soma</code>	(Mensagem curta)
	(Linha em branco)
<code>Criada uma nova função que aceita dois números e devolve a soma dos dois.</code>	(Mensagem detalhada)
<code>Aceita números negativos como input.</code>	
	(Linha em branco)
<code>Falta eliminar a repetição de código.</code>	(Problemas a resolver)

4 Aplicação de Comandos a um Projeto

A presente secção corresponde à ultima parte da formação. Consiste em criar e editar um projeto simples de forma a testar alguns dos comandos referidos anteriormente. No final da secção estará (após a formação), o *link* para o repositório usado.

Seguem-se alguns dos passos a seguir:

Parte 1:

1. Criar um repositório público no *GitHub*.
2. Clonar o repositório localmente num diretório à escolha.
3. Criar um ficheiro .txt simples e, sequencialmente, usar *git add*, *git commit*, *git push*.

Parte 2:

1. Criar uma *branch* nova (publicar no repositório remoto, eliminar e voltar a criar).
2. Criar uma função em *Python*.
3. Sequencialmente, usar *git add*, *git commit*, *git push*.

Parte 3:

1. Fazer uma pequena alteração *online* no *GitHub*.
2. *git fetch*, *git diff*, *git pull*
3. Efetuar uma *merge*

Parte 4:

1. Criar um *Commit* errado.
2. Usar *git revert*
3. Explorar o *git log*

Link do repositório usado: *Indisponível*